

SATA For Segmentation

Tanjeed Alam, Andrew Cheng
Stony Brook University

Abstract

In this project, we test the effectiveness of improving semantic segmentation models with the use of SATA, a method which in its original paper improved Vision Transformers' robustness and representation richness in the context of classification. We accomplished this by comparing performances between the baseline and the SATA-integrated models on both the clean and corrupted images sampled from the ADE20K scene parsing dataset. Our results overall showed that a naive implementation from SATA suffers from permutating the tokens in the ViT block. This is a unique consideration for segmentation models, which we attempt to address with our next model, which maintains token order through the SATA module. We found that SATA degraded performance in both clean and corrupted images, signaling a need to consider different adaptations of SATA.

1 Introduction

The goal of the project is to attempt integrating the module from the paper SATA: Spatial Auto-correlation Token Analysis for Enhancing the Robustness of Vision Transformers [1] into semantic segmentation models and gauge its impact on performance on both clean data, and corrupted data with various levels of noise. SATA is a plug-and-play module that doesn't require training, making it a versatile inclusion to models to improve robustness against corruptions.

Any improvements that can be gained with the SATA module are crucial for computer vision as one of its main benefits in the original paper was increased robustness against corruptions as shown by their benchmarks against the ImageNet-C dataset. In the real world, data is often far from perfect, so gains in performance on adversarial inputs have large amounts of practical importance.

Our main approach is to obtain baseline results for the EoMT model [2] on the clean ADE20K dataset [5], as well as sets of corrupted images. To eliminate potential confounding variables in the results, the corruptions were selected by using a library which exactly implements the corruptions used to produce the ImageNet-C dataset, which is what the SATA paper used to test robustness. Then, we will redo the same experiments, but with the SATA module implemented into our baseline models in the ViT Transformer blocks, and also test various choices of implementations with respect to the SATA module to compare the results and see which choices show promise.

Through our experiments, we found that a naive implementation of SATA completely destroys performance on segmentation tasks. This is likely due to the fact that SATA returns tokens in an arbitrary order, which is fine for the original task of classification, which is permutation invariant. So we made a version of SATA that keeps track of token order, and restores token position after merging. This change recovered a significant amount of performance, but was still significantly

below the baseline performance. We also tried computing the s-scores with cosine similarity, which was an idea mentioned in SATA [1], and increased performance on segmentation tasks. Finally, we realized that the layers we selected in EoMT had 4 register tokens, so we clipped those out from the SATA module. We found that this slightly increased performance with Attention s-scores, but didn't with Cosine s-scores.

2 Method

The input and output were as follows:

Given an input image $\mathbf{I}$ from the ADE20K Scene Parsing dataset, estimate a segmentation mask $\mathbf{M}$ where each pixel belongs to one of the 150 classes given within the dataset.

In particular, we wanted to observe the performance of a segmentation model with the SATA module integrated and compare it to the base model's performance using mIoU which we calculated using the same method as in the SC-CAM codebase [4].

The baseline model we chose was the Encoder-only Mask Transformer (EoMT) model.

2.1 SATA details

For a summary of the SATA module from the original paper, it gets placed into the ViT block right before the FFN. You compute the following things from your d-dimensional tokens x_i

$$\mathbf{x} = [x_1, x_2, \dots, x_n]$$

$$\mathbf{a} = [a_i = \frac{1}{d} \sum_{i=1}^d x_i(t)]$$

$$\mathbf{z} = \frac{\mathbf{a} - \mu}{\sigma}$$

Then your Moran's I metric is

$$I_l = [\text{diag}(\mathbf{z}\mathbf{z}^t W)]$$

Then the normalized version is

$$\mathbf{s} = \frac{\mathbf{I}_l - \mu_{I_l}}{\sigma_{I_l}}$$

Then we split into two sets $\mathbb{A}, \mathbb{B}$ where

$$\mathbb{A} = \{x_i : s_i < \alpha(\mu_s - |\hat{\mathbf{s}}|) \cup s_i > \alpha(\mu_s + |\hat{\mathbf{s}}|)\}$$

and then $\mathbb{B}$ is all tokens which were not selected for $\mathbb{A}$. Then we participate in the following matching process in the image below

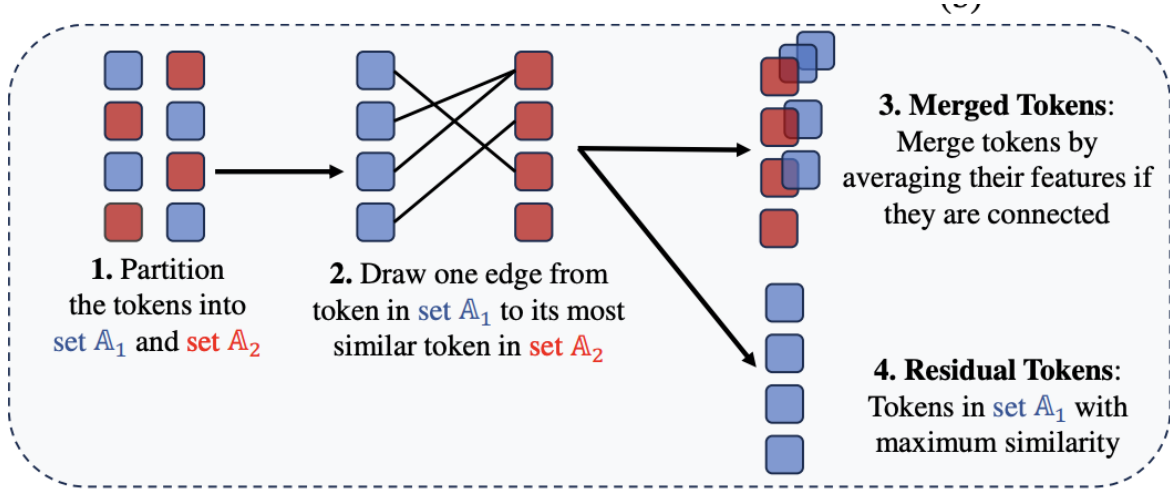


Figure 1: SATA matching

and we feed only the merged tokens and set $\mathbb{B}$ through the FFN and concatenate back the residual tokens to restore the original token count. For all the following models, the hyperparameters we chose were $\gamma = 0.7$ as in the original paper, which means that only the final 30% of transformer blocks were implemented with the SATA module. We also used $\alpha = 0.3$ for the matching process above as per the supplemental code. No other hyperparameter decisions were needed since no training was done in this project.

2.2 EoMT details

EoMT (Encoder only Mask Transformer) uses a DINOv2-pretrained ViT backbone with register tokens.

In the first 20 layers, it behaves like DINOv2 and takes patch embeddings of the images and passes them through ViT blocks with patch, CLS and register tokens.

Then at layer 21, a fixed set of learned query token embeddings is appended to the token sequence, with following layers attending over all tokens including the query tokens.

In the classification head, a single linear layer is applied to the final query token states, producing per-query class logits (including a no-object class).

Then in the mask head, the final query token states are used to produce per-query mask feature vectors. These are dot-producted with the upsampled patch token features to produce mask logits.

We apply SATA to layers 16-20, which corresponds to our choice of $\gamma = 0.7$, and avoiding the added query tokens in layers 21-24.

2.3 Our Implementations

For version 1 of the model, we simply implemented into EoMT the SATA module with no additional modifications.

For version 2 of the model, we keep track of positions throughout the whole merging process of SATA. If the N tokens are represented as an N -dimensional array, then before and after the SATA

algorithm, the goal is for any modified tokens to be restored to their original index. To do this, once the tokens are split into sets A and B, it is easy to keep track of where the tokens in set B are since they do not participate in merging. On the other hand, once set A becomes A1 and A2, in the supplied code, A1’s tokens match to their most similar token in A2, and they all merge into A2’s slots. A1 then passes through as residual so just like B, it is easy to maintain their original positions. The merged tokens are the size of A2, so once an arbitrary token merged[i] passes through the FFN, it should be restored to the same original index of A2[i].

For version 3 of the model, we noticed that since segmentation relies more specifically on the local features of a spatial token compared to classification, we instead use a cosine similarity matrix for W instead of SATA’s proposed self-attention matrix.

For versions 4 and 5 of the model, we realized that the layers in EoMT we were applying the SATA modules to, had 4 register tokens prepended. As a result, we remade models 2 and 3 into models 4 and 5 respectively by clipping these register tokens out of the SATA process.

3 Dataset and Experimental Setup

Dataset Choice: ADE20K Scene Parsing 150

For preprocessing, we corrupted the dataset using the corresponding corruptions as found in the SATA paper. For implementing the corruptions, we used the standard imagecorruptions library, and excluded the glass blur and fog corruptions due to outdated dependencies.

For our baselines, we ran the EoMT model with its pre-trained weights on the clean dataset, as well as our corrupted dataset. Then, we ran the same experiments but with the EoMT models modified with placements of the SATA module. These models varied in the way the SATA module operated within them, such as through changing the choice of W or by maintaining positions throughout the matching process.

The main metric which we collected was the mean Intersection over Union (mIoU), and used that to compare performance across various corruptions.

3.1 Implementation Details

The project was implemented in Python using OpenCV, NumPy, HuggingFace, PyTorch, Matplotlib, and imagecorruptions. The experiments were run using a Google Colab notebook on a T4 or A100 GPU.

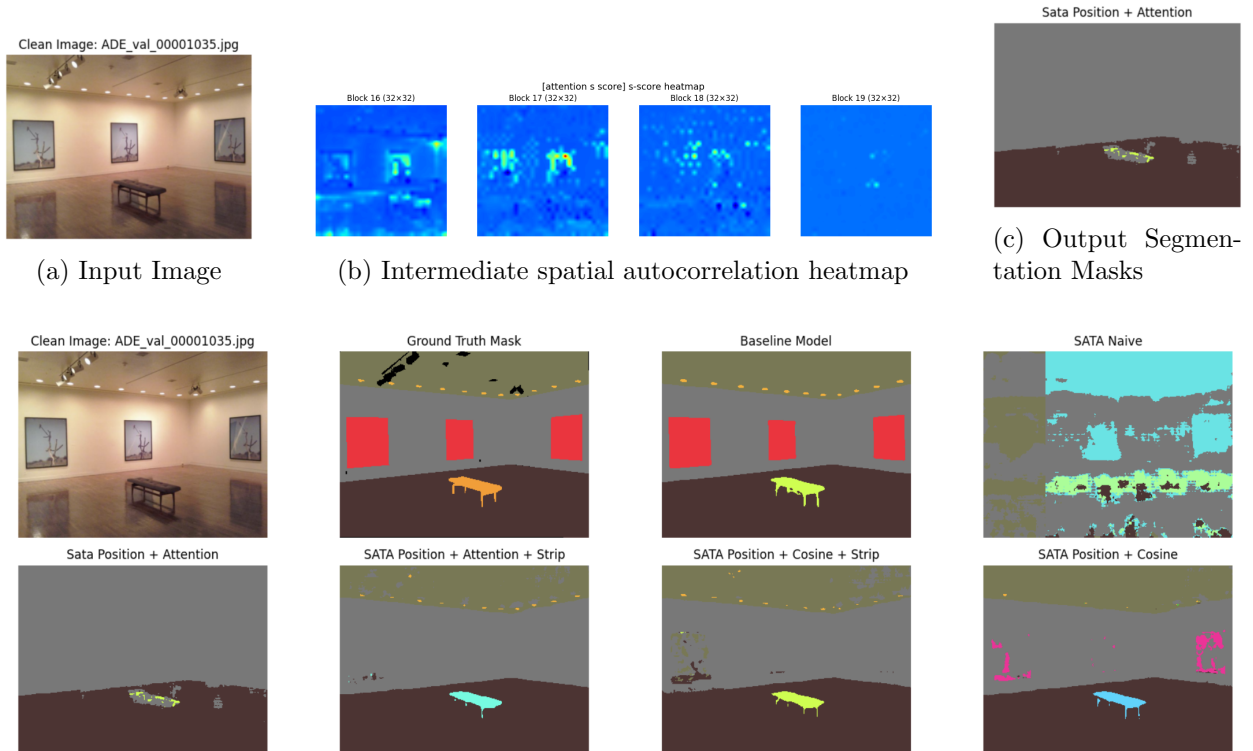
4 Results

As we will observe from the following results, SATA did not seem to prove very effective in enhancing the EoMT segmentation model.

4.1 Qualitative Results



Figure 2: Input Data. Top left is clean image, and the rest are corrupted versions.



(d) Ground truth mask, baseline output, and outputs across the variety of SATA models.

Figure 3: Overview of the SATA pipeline and segmentation results.

Figure 2 only shows the input, and the top of Figure 3 just shows an input, intermediate spatial autocorrelation map, and output for the model where we used attention = W and considered position in the token matching.

Most importantly, the bottom pictures in Figure 3 show the various outputs on a given image across our baseline and our SATA implementations. The biggest improvement in the output can be seen in the models once we start considering the positions of the tokens and maintaining them throughout the matching process. It is clear to see that none of our proposed SATA implementations were able to reach the original performance of the baseline model.

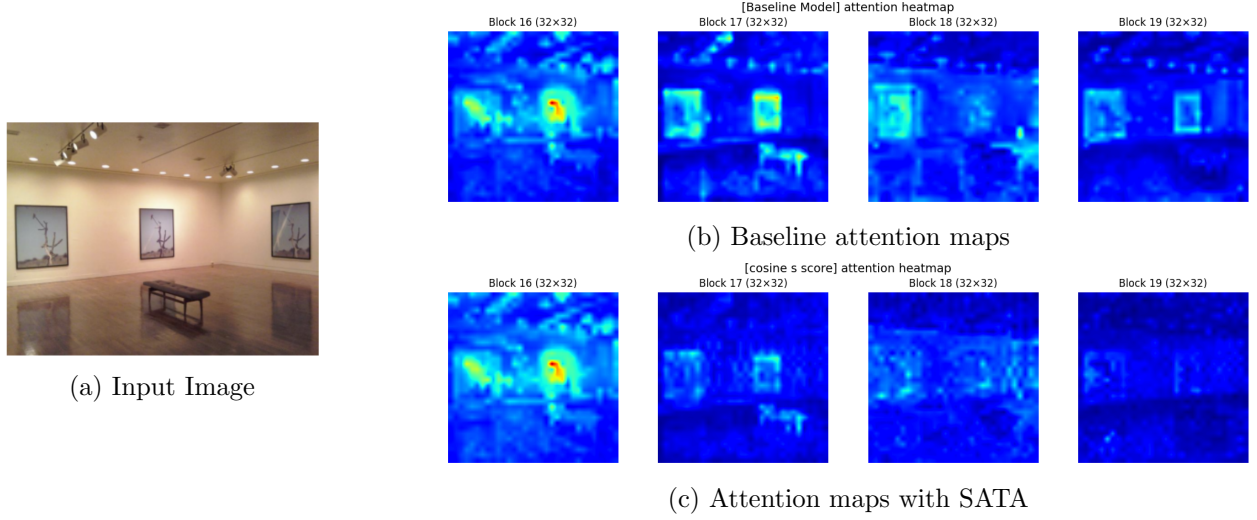


Figure 4: Comparison of attention maps

If we try to analyze the failure through the attention maps in the baseline versus the model with the cosine- W matrix as in Figure 4, we can see with the addition of the SATA module, the paintings in the baseline attention heatmap are more noticeable for all the blocks compared to the paintings in the SATA-modified model. This shows that it is possible that the SATA module is causing the model to lose important information as the paintings do not seem salient in the heatmaps generated by the SATA models. Losing this information is more detrimental in segmentation tasks than the original classification tasks since fine, locally-distinctive structures matter more for getting accurate masks.

4.2 Quantitative Results

Table 1: Quantitative results across corruption severities.

Method	Clean mIoU	Severity 1	Severity 2	Severity 3	Severity 4	Severity 5
Baseline	0.5463	0.5161	0.4925	0.4588	0.4168	0.3617
Naive Implementation	0.0253	0.0216	0.0234	0.0210	0.0192	0.0180
Attention + Positions	0.1655	0.1741	0.1573	0.1464	0.1279	0.1120
Attention + Positions + clipped register tokens	0.2082	0.2100	0.1999	0.1853	0.1672	0.1405
Cosine + Positions	0.2375	0.2309	0.2184	0.2009	0.1777	0.1442
Cosine + Positions + clipped register tokens	0.2146	0.2048	0.1921	0.1814	0.1660	0.1387

Table 2: Relative mIoU dropoff from clean to severity 5.

Method	Dropoff (clean – severity 5)/clean
Baseline	0.338
Naive Implementation	0.289
Attention + Positions	0.323
Attention + Positions + clipped register tokens	0.325
Cosine + Positions	0.393
Cosine + Positions + clipped register tokens	0.354

Overall, the results show that none of our proposed methods for SATA implementation were able to give a meaningful improvement as opposed to the baseline model in terms of MIOU. This can be seen as neither in the clean nor corrupt versions do the resulting MIOUs from the SATA-variant models become half as good as the baseline model.

We also can see from the dropoff table that none of the modified models were particularly effective at increasing robustness against corruptions as the dropoffs for each of the variant models are all around the baseline’s dropoff.

Limitations of our methods include that, due to limited compute, we were only able to run inference for $n = 200$ on the corrupted images, thus the results may be prone to variance. Another limitation is that we were unable to test on perhaps a more ideal model such as SegViT which has a simpler architecture, as we were unable to get pretrained weights on Google Colab due to library incompatibility.

5 Conclusion

In this project we assessed the effectiveness of the SATA module in semantic segmentation models, both in terms of overall performance, as well as robustness against corruptions.

We accomplished this by comparing baseline performances on pretrained versions of EoMT to multiple SATA-adapted versions, each varying in the method chosen to implement the SATA module with considerations made with respect to the change of the downstream task.

Overall, our results showed that an implementation of SATA as defined in the original paper is unlikely to work for segmentation models due to the fact that each spatial token matters much more when constructing pixel-level masks. Even when SATA maintains the permutation of tokens, mIoU significantly dropped on all tests compared to the baseline model. There is a much deeper emphasis on local information as compared to the original paper’s setting of classification which relies on global information and context.

In the future, one can use the findings presented here to see that the path towards a successful implementation of SATA cannot solely rely on the exact mechanisms presented in the original paper. However, future work on adaptation of SATA can look into different ways to leverage the token-level spatial autocorrelation score, be it with the cosine-similarity change we suggest, or the original paper’s attention matrix suggestion.

6 Individual Contributions

- **Tanjeed Alam:** Created the slides and report. Drafted report. Helped create visualizations from the model’s outputs, collaborated on model design decisions. Gave advice on the infrastructure for running the models.
- **Andrew Cheng:** Worked on setting up the datasets, models and running inference on both the clean and corrupted datasets with the EoMT model. Collaborated on slides and helped refine the report.

References

- [1] Nikzad, Nick, et al. SATA: Spatial autocorrelation token analysis for enhancing the robustness of vision transformers. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, 2025.
- [2] Kerssies, Tommie, et al. Your ViT is secretly an image segmentation model. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, 2025.
- [3] Ranftl, René, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [4] Chang, Yu-Ting, et al. Weakly-supervised semantic segmentation via sub-category exploration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [5] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, “Scene parsing through ADE20K dataset,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 633–641.